AlphaBrain v3 — AWS + Open Source + Agentic AI Engineering Plan (12–13 Weeks)

This comprehensive roadmap defines the 12–13 week modular engineering plan for AlphaBrain v3 — a full-stack, AWS-native, open-source-driven AI system built for production-grade scalability, observability, and modularity. The emphasis is on engineering infrastructure, orchestration, and deployment; model performance is secondary. LLM orchestration frameworks (LangChain, LangGraph, CrewAI, Evals, and Guardrails) are layered progressively across the pipeline to build real-world agentic AI systems.

Guiding Principles

• AWS-only infrastructure using S3, ECS/Fargate, RDS, Terraform, CloudWatch, IAM, and Secrets Manager.

• Fully open-source framework stack (LangChain, LangGraph, CrewAI, MLflow, Airflow, Milvus, FastAPI, Prometheus/Grafana).

• Engineering-first: infrastructure, pipelines, observability, and reproducibility over ML performance.

• Modular design: components are swappable (e.g., Milvus ↔ Weaviate, Airflow ↔ Prefect, LangChain ↔ LlamaIndex).

• Agentic AI integration: LangChain → LangGraph → CrewAI → Evals → Guardrails.

• Compliance and cost discipline: AWS Budgets, data licensing, and legal guardrails.

Week 0 – Governance, Setup & Safety

Goal: Project charter, ethics, cost limits, and AWS setup.

- - Define goals, success metrics, and data licensing checklist.
- - Setup AWS Budgets and IAM roles.
- - Initialize repo structure (src, data, infra, ci, docs).
- - Add non-financial-advice disclaimer.

Week 1 – Infrastructure as Code & CI/CD

Goal: Establish Terraform + GitHub Actions for IaC and automation.

- - Terraform setup for S3, ECS, RDS, IAM.
- - GitHub Actions for build/test/deploy.
- - Dockerfile templates for API/worker containers.
- - Provision MLflow bucket and artifact store.

Week 2 – Data Lake & Batch Ingestion

Goal: Batch pipelines on Airflow for financial + news data.

- - Airflow DAG for data ingestion into S3.
- - Postgres metadata catalog for data lineage.
- - Retry, logging, and versioned schema.

Week 3 – Data Versioning & Feature Engineering

Goal: Add DVC/Delta Lake for data versioning and features.

- - Integrate DVC with S3 backend.
- - Airflow DAG for feature computation.
- - Feature store stub (Feast/Postgres).
- - Unit tests for feature pipelines.

Week 4 – Embeddings & Vector DB

Goal: Build semantic layer with embeddings and retrieval.

- - SentenceTransformers for embeddings.
- - Milvus/pgvector vector DB integration.
- - LangChain retriever abstraction.
- - FastAPI retrieval endpoint.

Week 5 – Model Lifecycle + RAG Pipeline

Goal: Introduce MLflow, model registry, and RAG prototype.

- - MLflow server setup (Postgres + S3).
- - Baseline model training + registration.
- - LangChain RAG pipeline: retriever $\rightarrow$ LLM $\rightarrow$ summarizer.
- - API route for pipeline inference.

Week 6 – Inference Service & Deployment

Goal: Containerize inference and deploy via ECS.

- - FastAPI model service in ECS Fargate.
- - GitHub Actions auto-deploy to staging.
- - Load testing (Locust/k6).
- - LangChain Serve wrapper optional.

Week 7 – Observability & Monitoring

Goal: Add metrics, dashboards, and alerting.

- - Prometheus + Grafana stack.
- - Structured logging via CloudWatch.
- - SLOs: latency, availability, error rate.
- - Alerting setup for anomalies.

Week 8 – CI/CD for Pipelines & Security

Goal: Integrate security and automation.

- - Terraform checks in CI.
- - AWS Secrets Manager for credentials.
- - IAM least privilege enforcement.
- - LangChain DevOps agent for monitoring pipelines.

Week 9 – Retraining & LangGraph Orchestration

Goal: Automate model lifecycle with LangGraph flows.

- - Airflow retrain DAG (train → register → deploy).
- - LangGraph workflow orchestration.
- - Feature store consistency validation.
- - Rollback + versioning policy.

Week 10 – Streaming & Multi-Agent (CrewAI)

Goal: Add event-driven AI pipeline with CrewAI agents.

- - Kafka/Kinesis event ingestion.
- - CrewAI agents: Data, Embedding, Notification.
- - Streaming embeddings update in vector DB.
- - Integration test for async agent workflows.

Week 11 – Evaluation & Guardrails

Goal: LLM output evaluation and schema safety.

- - Integrate OpenAI Evals / DeepEval.
- - LangChain Guardrails for validation.
- - Evaluation metrics stored in MLflow.
- - Regression tests for RAG pipelines.

Week 12 – Final Integration & Demo

Goal: System integration, documentation, and demo.

- - End-to-end demo: ingest → train → deploy → infer.
- - Architecture diagram + runbooks.
- - Cost/compliance review and MVP sign-off.

Week 13 – Kubernetes & Scaling (Optional)

Goal: Optional: migrate to EKS for scale.

- - Terraform EKS cluster.
- - Deploy via Helm charts.
- - Horizontal Pod Autoscaling (HPA).
- - Cost optimization and monitoring.

Acceptance Criteria

• End-to-end data and model pipelines automated via Airflow + Terraform.

• Containerized inference API with observability and autoscaling on AWS.

• MLflow registry with tracked models and lineage.

• LangChain, LangGraph, CrewAI integrated into orchestration pipeline.

• Evals and Guardrails enforce LLM quality and safety.

• Reproducibility from code → infra → deployment.

Cost Optimization

• Use AWS Budgets and alerts for spend control.

• Run GPU jobs only when needed; prefer Spot instances.

• Auto-stop dev containers and RDS when idle.

• Use open-source models (Llama 3, Mistral) for inference.

• Compress logs and archive S3 data with lifecycle rules.

Prepared by: Assistant — AlphaBrain Project